

Venaball & Venaball Club

Ce qu'il reste à faire

Feuille de route détaillée, par ordre de priorité

Date	4 août 2026
Établi depuis	Lecture du code des deux dépôts + docs 24, 31, 33, 34, 36
Périmètre	mabb.fr · manager.mabb.fr · pirb.mabb.fr · app mobile Venaball
Échéance jury CDA	Avril 2027

À lire avant tout

Le cahier des charges est une **carte du territoire**, pas une liste de courses : il dit lui-même page 26 qu'il « n'engage pas à tout livrer en V1 ». Le cœur métier réellement utilisé par le club est à **~90 % et tourne en production**. Ce document ne liste donc pas « tout le CDC », mais **ce qui mérite vraiment d'être fait**, dans l'ordre où je le ferais.

Les trois choses à faire cette semaine

1. Sauvegarder la base de production (elle n'est sauvegardée nulle part).
2. Passer le dépôt GitHub en privé (données de mineures dans l'historique).
3. Authentifier le domaine mabb.fr en DKIM/SPF (sans ça, les mails ne partent pas).

Vue d'ensemble

Chaque ligne renvoie à une fiche détaillée dans les pages suivantes.

1	Sauvegarde de la base de production	CRITIQUE	2 h
2	Dépôt GitHub en privé	CRITIQUE	5 min
3	Authentification du domaine mail (DKIM/SPF)	CRITIQUE	1 h + délai DNS
4	Fichiers sensibles hors de public/	HAUTE	2 à 3 jours
5	Vérifier que le cron RGPD tourne	HAUTE	30 min
6	Tests automatisés (attendu du jury CDA)	HAUTE	2 à 3 semaines
7	Anti-force-brute sur le web	MOYENNE	1 h
8	Nettoyage du contenu CMS (XSS)	MOYENNE	3 h
9	En-têtes de sécurité HTTP	MOYENNE	1 h
10	Compte désactivé qui peut encore se connecter	MOYENNE	2 h
11	Promotion automatique des sessions Stats Live	MOYENNE	4 h
12	Fusion des deux agrégateurs de stats	MOYENNE	1 jour
13	Sortie de l'app sur les stores	HAUTE	1 à 2 semaines
14	Messagerie interne (seul trou métier du CDC)	MOYENNE	2 à 3 semaines
15	Polish fonctionnel (iCal, QR, relances, recherche)	BASSE	variable

Ce qui est volontairement hors périmètre

Boutique e-commerce (§3.7) : descopée pour MABB — club en quartier prioritaire, peu de moyens côté familles. Conservée comme fonctionnalité à vendre à d'autres clubs.

Plans et abonnements payants (§9), **compte unique Velito** (§6.4), **encaissement par carte** : volet commercial du SaaS, jamais commencé, assumé.

Bloc 1 — Les urgences

Ces trois points ne sont pas des fonctionnalités. Ce sont des risques ouverts : perte de données, exposition de données personnelles de mineures, et des e-mails qui ne partent pas. Rien d'autre ne devrait passer avant.

1. Sauvegarde de la base de production

CRITIQUE | 2 h

Pourquoi. Aujourd'hui, **aucune sauvegarde n'existe**. Une erreur de manipulation, une migration ratée ou un incident OVH, et le club perd toutes ses données : joueuses, licences, trésorerie, bilans. C'est le risque numéro un du projet, très au-dessus de n'importe quel pourcentage de cahier des charges.

Comment procéder :

- Se connecter en SSH et créer le dossier de sauvegarde (hors du dossier web) :

```
mkdir -p ~/backups && chmod 700 ~/backups
```

- Récupérer les identifiants de la base depuis `.env.local` (ne jamais les écrire ailleurs), puis tester un export manuel :

```
cd ~/mabb-site
mysqldump --single-transaction --quick \
-h HOTE -u UTILISATEUR -p BASE \
| gzip >> ~/backups/mabb_$(date +%Y%m%d).sql.gz
ls -lh ~/backups/
```

- Écrire un script `~/backups/backup.sh` qui fait la même chose et supprime les sauvegardes de plus de 30 jours (`find ~/backups -name "*.sql.gz" -mtime +30 -delete`).
- Programmer le script dans l'interface OVH : Hébergements → Tâches planifiées (cron), une fois par jour vers 4 h du matin.
- **Tester la restauration** sur une base vide. Une sauvegarde jamais restaurée n'est pas une sauvegarde, c'est une intention.
- Télécharger une copie hors du serveur (une panne OVH emporterait aussi les sauvegardes stockées chez OVH).

Comment vérifier que c'est fait. Un fichier `.sql.gz` apparaît chaque matin dans `~/backups/`, et tu as réussi à restaurer une de ces archives dans une base de test.

2. Passer le dépôt GitHub en privé

CRITIQUE | 5 min

Pourquoi. Le dépôt `Clavelclams/mabb-site` est public. Les fichiers sensibles ont été retirés du dernier commit, mais **l'historique git les contient toujours** : un compte rendu d'entretien nominatif et trois exports SQL avec des noms de joueuses. N'importe qui sachant chercher peut les lire. C'est une violation de la protection des données personnelles de mineures.

Comment procéder :

- GitHub → le dépôt → **Settings** → onglet **General** → tout en bas, **Danger Zone** → **Change repository visibility** → **Private**.
- Vérifier ensuite que le déploiement fonctionne toujours : le serveur utilise déjà une authentification, mais un `git fetch` de contrôle coûte 10 secondes.
- *Option plus lourde, seulement si le dépôt doit rester public* : réécrire l'historique avec `git filter-repo` pour effacer les fichiers, puis forcer la mise à jour. Cette opération casse tous les clones existants et impose de considérer les secrets passés comme compromis.

Comment vérifier que c'est fait. Le dépôt affiche l'étiquette « Private » sous son nom, et une navigation privée sur l'URL du dépôt renvoie une erreur 404.

Pourquoi. Le compte Brevo est créé et MAILER_DSN est en place, mais le domaine mabb.fr n'est pas authentifié. Résultat : les convocations, les invitations de parents et les réinitialisations de mot de passe partent en courrier indésirable, ou ne partent pas du tout. **C'est aussi le point qui casserait une démonstration devant le jury.**

Comment procéder :

- Dans Brevo : Senders & IP → Domains → ajouter mabb.fr. Brevo affiche les enregistrements DNS à créer (une clé DKIM, un enregistrement SPF, parfois un DMARC).
- Dans l'espace client OVH : Web Cloud → Noms de domaine → mabb.fr → **Zone DNS** → ajouter les enregistrements fournis, à l'identique.
- Attendre la propagation (de 15 minutes à 24 heures), puis cliquer « Vérifier » dans Brevo.
- Tester un envoi réel depuis le site :

```
php bin/console app:create-admin # ou un reset de mot de passe depuis /mot-de-passe-oublie
```

- Vérifier la réception **et** l'en-tête du message (dans Gmail : « Afficher l'original » → DKIM doit indiquer « PASS »).

Comment vérifier que c'est fait. Brevo affiche le domaine en vert, et un mail de test arrive dans la boîte de réception principale, pas dans les indésirables.

Bloc 2 — Protection des données et sécurité

L'audit du 26 juillet a corrigé trois failles critiques. Ce qui suit est le reste : moins urgent, mais réel. Le point 4 est le plus lourd et le plus important.

4. Sortir les fichiers sensibles de public/

HAUTE | 2 à 3 jours

Pourquoi. Sept uploaders sur huit écrivent dans `public/uploads/`, c'est-à-dire dans le dossier servi directement par Apache : justificatifs de trésorerie nominatifs, documents de réunion, feuilles de match, photos de joueuses mineures. Les contrôleurs vérifient bien les droits *avant d'afficher*, mais rien n'empêche d'aller chercher le fichier par son URL. Le fichier `.htaccess` posé le 26/07 est un pansement : il dépend d'Apache et d'une configuration que l'hébergeur peut changer.

Comment procéder :

- **Le modèle à suivre existe déjà dans le projet :** `DechargeSortieUploader` écrit dans `var/decharges/` et le fichier est servi par un contrôleur qui vérifie les droits. Copier ce fonctionnement.
- Traiter les uploaders par ordre de sensibilité : justificatifs de trésorerie et de notes de frais d'abord, puis documents de réunion, puis feuilles de match, puis photos.
- Pour chacun : changer le dossier de destination vers `var/uploads/<type>/`, créer ou réutiliser une route de téléchargement protégée, remplacer les liens directs dans les templates.
- Remplacer au passage `uniqid()` par `bin2hex(random_bytes(16))` dans les noms de fichiers : `uniqid()` contient l'horodatage, donc un nom voisin se devine.
- **Prévoir la migration des fichiers déjà en place :** une commande qui déplace l'existant et met à jour les chemins en base. Ne pas laisser deux emplacements coexister.

Comment vérifier que c'est fait. Une URL directe vers un justificatif renvoie 404, alors que le téléchargement depuis l'application fonctionne toujours.

5. Vérifier que le cron RGPD tourne réellement

HAUTE | 30 min

Pourquoi. La purge des données de mineures ayant quitté le club (`app:sorties:purger-rgpd`) et la clôture hebdomadaire des affectations de match dépendent d'un cron OVH **qui n'est pas versionné dans le dépôt**. Personne n'a jamais confirmé qu'il s'exécute. S'il ne tourne pas, la purge n'a jamais eu lieu et le club conserve des données qu'il devrait avoir effacées.

Comment procéder :

- Espace client OVH → Hébergements → `mabb.fr` → onglet **Tâches planifiées**. Regarder ce qui est déclaré et la date de dernière exécution.
- S'il n'y a rien, créer les deux tâches :

```
php ~/mabb-site/bin/console app:sorties:purger-rgpd --env=prod # 1x/semaine
php ~/mabb-site/bin/console app:otm:cloturer --env=prod # tous les mercredis
```

- **Documenter ces crons dans le dépôt** (par exemple `instruction/33_SAUVEGARDE_BDD.md` ou un fichier dédié) : aujourd'hui, cette configuration n'existe que dans l'interface OVH et disparaîtrait avec le compte.
- Lancer une fois la commande à la main pour voir ce qu'elle produit.

Comment vérifier que c'est fait. La tâche apparaît dans l'interface OVH avec une date de dernière exécution récente, et un fichier du dépôt décrit ce qui est planifié.

7. Limiter les tentatives de connexion sur le web

MOYENNE | 1 h

Pourquoi. Le formulaire de connexion web n'a **aucune limite de tentatives** : les échecs sont journalisés et alertés, mais pas bloqués. L'API mobile, elle, est protégée depuis le 13 juillet. Les blocs de configuration sont

déjà écrits dans `security.yaml`, **mis en commentaire**.

Comment procéder :

- **Attention, l'ordre compte.** Décommenter sans installer le composant empêche le conteneur Symfony de compiler : **tout le site tombe en erreur 500**. Rappel : `vendor/` n'est pas versionné et le déploiement ne lance pas Composer.

- Sur le poste de développement :

```
composer require symfony/rate-limiter
git add composer.json composer.lock
git commit -m "chore: ajoute symfony/rate-limiter"
git push origin main
```

- Sur le serveur, **avant** de décommenter quoi que ce soit :

```
cd ~/mabb-site
git fetch origin && git reset --hard origin/main
composer install --no-dev --optimize-autoloader
rm -rf var/cache/prod && php bin/console cache:clear --env=prod
```

- Vérifier que le site répond toujours, **puis seulement** décommenter les quatre blocs `login_throttling`, `committer`, redéployer.

Comment vérifier que c'est fait. Cinq échecs de connexion sur le même compte, et la sixième tentative renvoie une erreur 429 au lieu du formulaire.

8. Nettoyer le contenu éditorial affiché en brut

MOYENNE | 3 h

Pourquoi. Six templates affichent du contenu du CMS avec le filtre `|raw`, c'est-à-dire sans échapper le HTML. C'est nécessaire (l'éditeur produit du HTML), mais aujourd'hui rien ne filtre ce qui passe. Un compte ayant accès à l'administration éditoriale peut injecter du JavaScript qui s'exécutera chez tous les visiteurs.

Comment procéder :

- Installer le composant officiel de nettoyage :

```
composer require symfony/html-sanitizer
```

- Le configurer dans `config/packages/html_sanitizer.yaml` en autorisant ce dont l'éditeur a besoin (titres, gras, italique, listes, liens, images) et rien d'autre.
- Remplacer dans les six templates : `{{ contenu|raw }}` devient `{{ contenu|sanitize_html }}`.
- Relire chaque page concernée : un nettoyage trop strict casse la mise en forme existante. Ajuster la configuration plutôt que de revenir en arrière.
- **Ne pas oublier** le `composer install --no-dev` sur le serveur, comme pour le point 7.

Comment vérifier que c'est fait. Un article contenant une balise de script s'affiche sans exécuter le script, et la mise en forme normale des articles existants est intacte.

9. Ajouter les en-têtes de sécurité HTTP

MOYENNE | 1 h

Pourquoi. Les réponses ne portent aucun en-tête de sécurité. Ce sont trois lignes qui bloquent des classes entières d'attaques : intégration du site dans une iframe piégée, interprétation d'un fichier comme un type différent de ce qu'il est, fuite de l'URL visitée vers des sites tiers. Recommandé dès le 13 juillet, toujours absent.

Comment procéder :

- Créer un écouteur d'événement, par exemple `src/EventListener/SecurityHeadersListener.php`, branché sur `kernel.response`.
- Y poser trois en-têtes :

```
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
```

- **Exception à prévoir :** les pages du Playground sont chargées dans une WebView par l'application mobile. Vérifier que `X-Frame-Options: DENY` ne les casse pas ; si c'est le cas, exclure ces routes.
- *Une politique de sécurité de contenu complète (CSP) est un chantier à part :* les templates utilisent du style en ligne partout, une CSP stricte casserait l'affichage. À traiter plus tard, séparément.

Comment vérifier que c'est fait. Dans les outils de développement du navigateur, onglet Réseau, les trois en-têtes apparaissent sur les réponses. Le Playground fonctionne toujours dans l'app.

10. Un compte désactivé peut encore se connecter

MOYENNE | 2 h

Pourquoi. Le champ `isActive` existe sur les comptes, mais rien ne le vérifie au moment de la connexion : désactiver un compte ne l'empêche pas d'entrer. Pire, ses jetons d'application mobile **survivent à un effacement RGPD** : les données sont anonymisées, l'accès reste ouvert.

Comment procéder :

- Créer `src/Security/UserChecker.php` implémentant `UserCheckerInterface`, et refuser la connexion quand `isActive` est faux, avec un message neutre (ne pas révéler si le compte existe).
- Le déclarer sur **chacun** des firewalls concernés dans `security.yaml` — c'est l'erreur classique de n'en configurer qu'un.

- Côté API : révoquer les jetons à la désactivation et à l'anonymisation RGPD. Dans RgpdAnonymizer, supprimer les ApiToken et PushToken du compte.
- Vérifier aussi que le contrôle du jeton d'API teste l'état du compte à chaque requête, pas seulement à la connexion.

Comment vérifier que c'est fait. Un compte désactivé se voit refuser la connexion sur les trois espaces web, et son application mobile est déconnectée à la requête suivante.

Bloc 3 — Ce que le jury CDA attend

Mon avis franc

Le pourcentage du cahier des charges n'est pas ce qui compte devant un jury. Un développeur seul qui livre du multi-tenant, de la saisie temps réel, une API et une application native, **en production avec de vrais utilisateurs**, est déjà très au-dessus de l'attendu. Ce qui manque n'est pas fonctionnel : **ce sont les tests**. Si tu ne dois faire qu'une seule chose de ce bloc dans les prochaines semaines, c'est le point 6.

6. Tests automatisés

HAUTE | 2 à 3 semaines

Pourquoi. Le dossier tests/ est quasi vide. Or la couverture de tests est un **bloc explicitement attendu au référentiel CDA**, et c'est la première question technique qu'un jury pose sur un projet de cette taille. C'est aussi ce qui te permettra de modifier le code sans peur, sur un projet qui tourne en production.

Comment procéder :

- Ne pas viser une couverture complète : viser **les zones où un bug coûte cher**, et savoir expliquer pourquoi tu as choisi celles-là. C'est ce discours que le jury évalue.
- **Priorité 1 — l'isolation multi-tenant.** Des tests fonctionnels qui vérifient qu'un utilisateur du club A reçoit bien une erreur en accédant aux données du club B. C'est le risque le plus grave du projet, et trois failles de ce type ont déjà été trouvées.
- **Priorité 2 — les calculs métier**, en tests unitaires purs, sans base de données : minutes jouées, évaluation FIBA, calcul de saison, calcul de catégorie d'âge. Rapides à écrire, faciles à défendre.
- **Priorité 3 — l'authentification de l'API** : jeton valide, jeton expiré, jeton révoqué, limitation des tentatives.
- Mise en route :

```
composer require --dev symfony/test-pack
php bin/console make:test # TestCase ou WebTestCase
php bin/phpunit
```

- Utiliser une base de test séparée (.env.test) et des fixtures dédiées. Ne jamais faire tourner les tests sur la base de production.
- Viser une trentaine de tests solides plutôt que trois cents superficiels.

Comment vérifier que c'est fait. php bin/phpunit est vert, et tu peux expliquer en deux minutes pourquoi tu as testé ces zones-là plutôt que d'autres.

Les autres attendus du jury, déjà couverts

- **Documentation** : dossier instruction/ (37 documents) et les pages Notion de cartographie. Solide.
- **Modèle de données** : 67 entités, commande app:puml pour générer le diagramme. À relancer avant la soutenance.
- **Sécurité** : deux passes d'audit documentées avec les failles trouvées et corrigées. C'est un excellent sujet de soutenance — montre une démarche, pas seulement un résultat.
- **Déploiement** : procédure écrite, contraintes de l'hébergement mutualisé comprises et documentées.
- **Accessibilité** : jamais évaluée. Un audit rapide (contrastes, navigation au clavier, attributs alt) serait un plus, sans être bloquant.

Bloc 4 — Fiabilisation des statistiques

Les deux gros défauts (minutes jouées fausses, titulaires faux) ont été corrigés le 26 juillet. Il reste deux points, dont un qui a un impact quotidien.

11. Promotion automatique des sessions Stats Live

MOYENNE | 4 h

Pourquoi. Une saisie de match ne remonte sur les fiches des joueuses que lorsqu'une session est **promue « officielle »**. Or, au 4 août, **aucune session n'a jamais été promue en production** : autrement dit, aucune donnée saisie en direct n'est jamais arrivée jusqu'aux joueuses. Le travail des bénévoles à la table de marque est perdu.

Comment procéder :

- **Réflexe immédiat, sans code** : intégrer le geste dans les habitudes du club. Après chaque match, ouvrir la session et cliquer « Promouvoir en officielle ». C'est ce qui déclenche tout le reste.
- Reprendre les matchs déjà saisis mais jamais promus : les promouvoir un par un, les stats se génèrent alors avec le calcul corrigé.
- **Amélioration à coder** : quand il n'existe qu'une seule session pour une rencontre, la promouvoir automatiquement à la clôture du match. L'ambiguïté n'existe que lorsque plusieurs bénévoles ont saisi en parallèle.
- Compléter par une alerte sur le tableau de bord : « 3 matchs saisis mais non validés », avec un lien direct. Une donnée saisie et jamais publiée est le pire des deux mondes.

Comment vérifier que c'est fait. Après un match saisi en direct, les minutes et les points apparaissent sur la fiche de la joueuse sans intervention manuelle.

12. Fusionner les deux agrégateurs de statistiques

MOYENNE | 1 jour

Pourquoi. `EvaluationCalculator` et `JoueurStatsAggregator` font un travail très proche : agréger des évaluations de match en moyennes. Deux implémentations séparées signifient deux endroits où corriger un bug, et une divergence garantie à terme entre ce qu'affiche le web et ce que renvoie l'API.

Comment procéder :

- Comparer les deux classes et lister ce que chacune fait que l'autre ne fait pas.
- Choisir la plus complète comme socle, y ajouter ce qui manque, puis faire pointer l'ensemble des appelants vers elle.
- **Écrire les tests d'abord** : ce refactoring touche des chiffres affichés aux familles. Des tests unitaires sur les moyennes rendent l'opération sûre — et c'est une pierre de plus pour le point 6.
- Supprimer l'ancienne classe seulement une fois qu'aucune référence ne subsiste (`grep -rn "JoueurStatsAggregator" src/`).
- Profiter du passage pour aligner le filtrage par source de données (`live / ffbb / manuel`) : l'API mobile agrège encore toutes sources confondues, alors que le web les sépare depuis le 26 juillet.

Comment vérifier que c'est fait. Une seule classe d'agrégation subsiste, les tests passent, et le web et l'application affichent les mêmes chiffres pour la même joueuse.

Bloc 5 — Sortir l'application mobile

L'application est codée à environ 95 %. **Ce qui bloque n'est plus du développement** : ce sont des démarches administratives, puis un test de terrain.

13. Publication sur l'App Store et Google Play

HAUTE | 1 à 2 semaines

Pourquoi. Sans comptes développeur, il n'y a ni certificat de notification, ni build distribuable, ni TestFlight. Tout le reste en découle. Le numéro D-U-N-S, qui était le point long (plusieurs jours d'attente), est **déjà obtenu**.

Comment procéder :

- **Étape 1 — les comptes, en organisation.** Apple Developer Program (99 \$/an) et Google Play Console (25 \$ une fois). Choisir « organisation » et non « particulier » : c'est ce qui permet d'afficher VENA comme éditeur, et cela exempte des 14 jours de test fermé imposés par Google aux comptes personnels. Compter environ 30 minutes de formulaires.
- **Étape 2 — le domaine.** Acheter venaball.fr : Apple exige une adresse de site d'entreprise vérifiable.
- **Étape 3 — le premier build.**

```
cd "Pirb store"  
eas build --profile development --platform android
```

- C'est ce build qui **active les notifications push** : le code existe déjà, il attend seulement un vrai binaire.
- **Étape 4 — le test au gymnase.** C'est l'étape la plus importante et la plus négligée. Rien n'a jamais été validé sur un vrai téléphone : la chaîne convocation (coach convoque → l'app affiche → la joueuse répond → le coach voit la réponse), le push, et la détection caméra en conditions réelles (lumière du gymnase, distance, arceau).
- **Étape 5 — le consentement parental.** À cadrer avant soumission : les utilisatrices sont mineures, les deux stores le vérifient. Passer par le club (le club recueille l'autorisation à l'inscription) est la voie la plus simple.
- **Étape 6** — TestFlight côté Apple, piste fermée côté Google, puis soumission.

Comment vérifier que c'est fait. L'application s'installe sur un téléphone qui n'est pas le tien, une convocation créée dans Manager y déclenche une notification, et la joueuse peut répondre.

Ce qu'on assume, les yeux ouverts

La feuille de route de juin interdisait de sortir une application native avant **100 utilisatrices actives**. On sortira avec zéro. C'est défendable : le Playground caméra n'existait pas dans ce plan, et c'est lui qui justifie l'application. Mais la conséquence doit être dite : **le premier téléphone testé au gymnase n'est pas une formalité de packaging, c'est le premier vrai test produit**. Prévoir que des choses ne marcheront pas.

Bloc 6 — Ce qui manque au cahier des charges

14. Messagerie interne

MOYENNE | 2 à 3 semaines

Pourquoi. C'est le **seul vrai trou métier** du cahier des charges (§4.14 côté Manager, §5.10 côté espace joueuse) : entièrement à faire, zéro ligne de code. Aujourd'hui, la communication passe par les notifications (à sens unique) et les e-mails.

Comment procéder :

- **Avant de coder, poser la question qui tue** : le club en a-t-il besoin, ou utilise-t-il déjà WhatsApp ? Construire une messagerie que personne n'utilisera parce que les groupes WhatsApp existent déjà serait deux à trois semaines perdues. **Demander aux coachs et à la secrétaire avant d'écrire la première ligne.**
- Si le besoin est confirmé, commencer **petit et utile** : messages du staff vers une équipe, non conversationnels (une annonce, un accusé de lecture). Cela couvre 80 % du besoin réel pour 20 % du travail.
- Ne pas partir sur du temps réel : l'hébergement mutualisé n'a ni WebSocket, ni Redis, ni worker. Un rafraîchissement classique suffit largement pour des annonces.
- Points de vigilance : ce sont des **mineures**. Prévoir la modération, la conservation et l'effacement des messages dans le registre RGPD, et interdire le message privé adulte → mineure sans traçabilité.
- Entités probables : Message, MessageDestinataire (avec date de lecture), multi-tenant par `club_id` comme le reste.

Comment vérifier que c'est fait. Un coach envoie une annonce à son équipe, les joueuses la voient sur le web et dans l'application, et le coach sait qui l'a lue.

15. Compléments fonctionnels

BASSE | variable

Pourquoi. Aucun n'est bloquant. Ils sont classés ici du plus utile au plus accessoire, avec le temps approximatif de chacun.

Comment procéder :

- **Relance de convocation à J-3** (§4.4) — une commande console qui envoie un rappel aux joueuses n'ayant pas répondu, branchée sur le cron existant. Fort impact, faible coût. *Une demi-journée.*
- **Alerte d'expiration des documents** (§4.9) — certificats médicaux et licences arrivant à échéance, affichés sur le tableau de bord du secrétariat. *Une demi-journée.*
- **Export iCal** (§4.3, §5.3) — permet d'ajouter le calendrier des matchs et entraînements dans le téléphone. Confirmé absent. Bibliothèque simple, forte valeur perçue. *Un jour.*
- **Calendrier dynamique et fiches par équipe sur la vitrine** (§3.3, §3.5) — les données existent déjà en base, il s'agit de les afficher côté public. *Un à deux jours.*
- **Tarifs et pièces à fournir sur la page de pré-inscription** (§3.6) — complète le formulaire public déjà en ligne. *Deux heures.*
- **Recherche globale** (§4.19) — une barre unique qui cherche joueuses, équipes, rencontres. Confort réel quand l'effectif grossit. *Deux à trois jours.*
- **Carte de membre numérique avec QR code** (§5.8) et **pointage par QR** (§4.5) — séduisant en démonstration, utilité réelle à vérifier auprès du club avant de le construire. *Trois à quatre jours.*
- **Indicateurs d'impact en PDF automatique** (§4.13) — utile pour les dossiers de subvention, à cadrer avec les besoins réels du bureau. *Deux jours.*
- **Lot D du module Sorties** — registre RGPD et purge de fin de saison. Recoupe le point 4, à traiter en même temps. *Un jour.*

Dans quel ordre s'y prendre

Une proposition de séquence, pensée pour réduire le risque d'abord, puis servir la certification, puis enrichir le produit.

Cette semaine — supprimer les risques ouverts

- Sauvegarde de la base **(1)** — deux heures qui protègent tout le reste.
- Dépôt en privé **(2)** — cinq minutes.
- DKIM/SPF **(3)** — lancer la procédure tôt, la propagation DNS prend du temps.
- Vérifier le cron RGPD **(5)** — trente minutes.

À la fin de la semaine, plus aucun risque de perte ou d'exposition de données ne reste ouvert. C'est le seul jalon vraiment non négociable.

Les deux semaines suivantes — l'usage quotidien

- Promotion des sessions **(11)** — d'abord le réflexe humain, puis l'automatisation. Sans cela, Stats Live ne sert à rien.
- Sortir les fichiers sensibles de public/ **(4)** — le plus gros morceau du bloc sécurité.
- En-têtes HTTP **(9)** et anti-force-brute **(7)** — rapides, à grouper dans un même déploiement puisque les deux touchent la configuration.

Le mois suivant — la certification

- Tests automatisés **(6)** — le meilleur investissement pour le jury, et pour ta tranquillité sur un projet en production.
- Fusion des agrégateurs **(12)** — à faire pendant cette période, les tests écrits juste avant rendent le refactoring sûr.
- Nettoyage du CMS **(8)** et vérification des comptes désactivés **(10)** — finir le bloc sécurité.

Quand tu veux — le produit

- Sortie de l'application **(13)** — à lancer dès que possible en parallèle : les démarches administratives ont leur propre rythme, indépendant du code.
- Compléments fonctionnels **(15)** — piocher selon les retours du club. La relance de convocation et les alertes d'expiration sont les plus rentables.
- Messagerie **(14)** — **seulement après avoir vérifié que le club en a besoin.**

Le mot de la fin

Ce document liste beaucoup de choses, mais l'essentiel tient en peu de lignes. Le produit **fonctionne et sert déjà un club réel** — c'est le plus dur, et c'est fait. Ce qui reste se range en trois catégories : **des risques à fermer** (sauvegarde, données personnelles), **une exigence de certification** (les tests), et **des envies de produit** (l'application, la messagerie, le confort). Ne pas confondre les trois, et traiter la première avant les autres.

Sources. Ce document est établi depuis le code des deux dépôts au 4 août 2026 et les documents `instruction/24` (suivi du cahier des charges), `31` (état réel du code), `33` (sauvegarde), `34` (audit de sécurité) et `36` (inventaire des fonctionnalités). En cas d'écart, le dépôt fait foi.